

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN HỌC
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
IT003.Q21.CTTN

SO SÁNH THỜI GIAN CHẠY CỦA CÁC THUẬT TOÁN SẮP XẾP

Sinh viên thực hiện
BÙI THÁI SƠN

MSSV
25521588

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 2 NĂM 2025

1 Giới thiệu

Báo cáo sử dụng chương trình python "gen.py" để tạo ra bộ dữ liệu "dataset.txt" theo yêu cầu:

- Hàng 1 là dãy số nguyên tăng dần
- Hàng 2 là dãy số nguyên giảm dần
- Hàng 3 đến 5 là dãy số nguyên không theo thứ tự
- Hàng 6 đến 10 là dãy số thực không theo thứ tự

Mỗi dãy số gồm một triệu (1000000) phần tử nằm trong khoảng $[0, 1000000000]$

Để chạy thuật toán và thống kê lại dữ liệu, báo cáo sử dụng chương trình python "benchmark.py". Chương trình bao gồm mã nguồn cách cài của 4 thuật toán: QuickSort, HeapSort, MergeSort và hàm sort của Numpy, và một hàm để so sánh (benchmark) thời gian hoạt động của các thuật toán. Chương trình sẽ xuất ra một bảng dữ liệu thống kê thời gian chạy của từng thuật toán, tương ứng với từng hàng trong bộ dữ liệu.

Bộ dữ liệu "dataset.txt" được sử dụng để viết báo cáo không được tải lên github do kích thước quá lớn.

Biểu đồ cột "Chart.png" được tạo bởi ứng dụng Gemini từ kết quả chạy của chương trình.

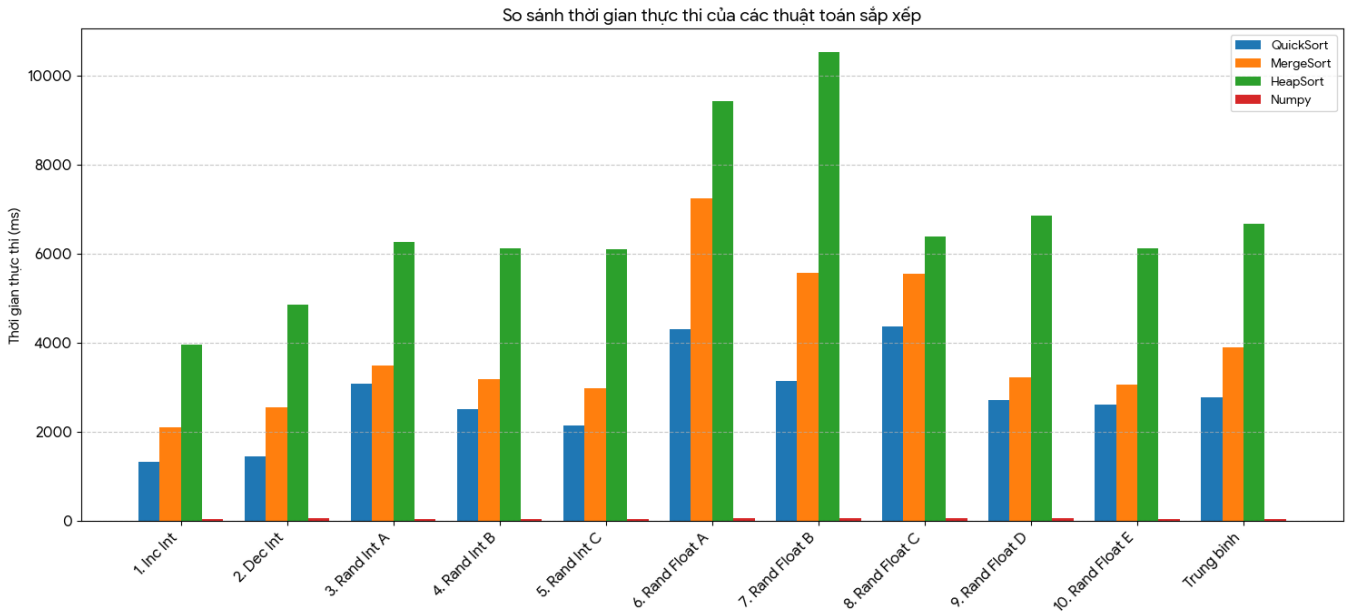
Tất cả các file liên quan được sử dụng trong bài báo cáo đều được đăng tải lên github: https://github.com/BetterCuckChuck/IT003.Q21.CTTN_LAB_REPORT_1.

2 Thống kê

Bảng 1: Kết quả thử nghiệm thời gian thực thi các thuật toán sắp xếp (ms)

Dãy Dữ Liệu	QuickSort	MergeSort	HeapSort	Numpy
1. Inc Int	1321	2114	3969	47
2. Dec Int	1449	2554	4861	63
3. Rand Int A	3088	3492	6267	42
4. Rand Int B	2517	3196	6134	45
5. Rand Int C	2148	2993	6100	38
6. Rand Float A	4318	7245	9442	61
7. Rand Float B	3139	5581	10530	71
8. Rand Float C	4380	5547	6390	60
9. Rand Float D	2721	3219	6868	67
10. Rand Float E	2606	3068	6126	39
Trung bình	2769	3901	6669	53

Hình 1: Biểu đồ cột cho thời gian thực thi các thuật toán sắp xếp



3 Phân tích, Đánh giá

Từ kết quả thực nghiệm được ghi nhận, ta có thể rút ra các phân tích chi tiết sau:

3.1 Về mặt tổng quát

- **Numpy Sort vượt trội nhất:** Thời gian chạy của Numpy luôn duy trì ở mức cực thấp (trung bình 53 ms), nhanh hơn gấp 50 đến hơn 100 lần so với các thuật toán tự cài đặt bằng Python thuần.

Nguyên nhân: Thư viện Numpy được viết bằng ngôn ngữ C và thực thi các phép toán trên mảng ở mức thấp (low-level). Điều này giúp thuật toán loại bỏ hoàn toàn chi phí (overhead) của trình thông dịch Python cũng như tối ưu hóa được việc quản lý bộ nhớ.

- **QuickSort nhanh nhất trong nhóm Python thuần:** QuickSort đạt tốc độ trung bình tốt nhất trong ba thuật toán thủ công (khoảng 2769 ms).

Nguyên nhân: Việc cài đặt QuickSort bằng kỹ thuật phân hoạch tại chỗ (in-place partitioning) giúp thuật toán không phải tạo ra các mảng con mới. Điều này tiết kiệm đáng kể chi phí cấp phát bộ nhớ và thời gian sao chép dữ liệu. Việc chọn Pivot ngẫu nhiên cũng giúp thuật toán tránh được trường hợp xấu nhất.

- **MergeSort nằm ở mức giữa:** MergeSort mất trung bình khoảng 3901 ms để hoàn thành việc sắp xếp dữ liệu ngẫu nhiên.

Nguyên nhân: Mặc dù thuật toán đảm bảo độ phức tạp thời gian ổn định là $O(N \log N)$, nhưng cách cài đặt đệ quy kết hợp với việc cắt mảng (list slicing) và tạo mảng mới trong

quá trình gộp (merge) đã tạo ra một lượng chi phí quản lý bộ nhớ khá lớn, khiến nó chậm hơn QuickSort.

- **HeapSort có thời gian chậm nhất:** HeapSort mất trung bình khoảng 6669 ms, có khoảng cách khá xa so với hai thuật toán còn lại.

Nguyên nhân: Việc cài đặt thủ công hàm `heapify` bằng Python thuần yêu cầu một lượng lớn các phép toán tính toán chỉ số, so sánh và hoán đổi phần tử liên tục qua các vòng lặp và đệ quy. Trong môi trường thông dịch như Python, các thao tác này sinh ra chi phí tính toán (overhead) rất lớn.

3.2 Về ảnh hưởng của dữ liệu đầu vào

- **Dữ liệu đã sắp xếp (Inc/Dec Int):** Cả QuickSort và MergeSort đều chạy nhanh hơn đáng kể trên dữ liệu đã có thứ tự hoặc ngược thứ tự (giảm từ mức $\sim 2500 - 3000$ ms xuống còn $\sim 1300 - 2500$ ms). Điều này cho thấy cơ chế chọn Pivot ngẫu nhiên và quá trình gộp hoạt động cực kỳ hiệu quả. Ngược lại, HeapSort vẫn mất nhiều thời gian do phải thao tác xây dựng lại toàn bộ cấu trúc cây bất kể thứ tự ban đầu.
- **Dữ liệu Int vs. Float:** Thực nghiệm cho thấy có sự chênh lệch rõ rệt về thời gian xử lý khi thay đổi kiểu dữ liệu. Các thuật toán chạy trên tập dữ liệu số Thực (Rand Float) đều tốn nhiều thời gian hơn so với tập số Nguyên (Rand Int). Chi phí thực hiện phép so sánh và hoán đổi đối với kiểu dữ liệu thực tốn kém tài nguyên CPU hơn, thể hiện rõ rệt nhất ở HeapSort (thời gian tăng vọt lên tới ~ 10530 ms).

4 Kết luận

- **Về mặt ứng dụng thực tế:** Nếu yêu cầu tốc độ xử lý cao trên các hệ thống thực tế sử dụng Python, việc tận dụng các thư viện tối ưu sẵn như `Numpy` hoặc hàm built-in `sorted()` là bắt buộc để đạt hiệu năng tốt nhất thay vì tự cài đặt lại thuật toán.
- **Về mặt thuật toán thủ công:** Trong mục đích học thuật và tự cài đặt bằng mã nguồn Python thuần, QuickSort là lựa chọn mang lại hiệu năng cao nhất.